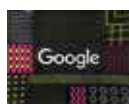




Zig-zag foldable phone

TCL revealed the TCL-flex, a zig-zag foldable phone which opens up to become a 10-inch tablet. <https://dgit.in/nov19-47>



New .new!

Google has added yet another .new domain, simply type "cal.new" or "meeting.new" to reach a blank calendar entry. <https://dgit.in/nov19-48>

```
service hello on new
http:Listener(9090) {
    # A resource representing an invocable API method
    # accessible at `/hello/sayHello`.
    #
    # + caller - the client invoking this resource
    # + request - the inbound request
    resource function
    sayHello(http:Caller caller,
    http:Request request) {
        // Sends a response back to the caller.
        error? result = caller-
        >respond("Hello Ballerina!");
```

client. If you use cURL, then the API method is available at /hello/sayHello

```
$ curl http://localhost:9090/hello/sayHello
```

You will get the following output:

Hello Ballerina!

It's quite simple if you think of your application in the form of microservices, each of which function independently.

WHAT MAKES BALLERINA SO DIFFERENT?

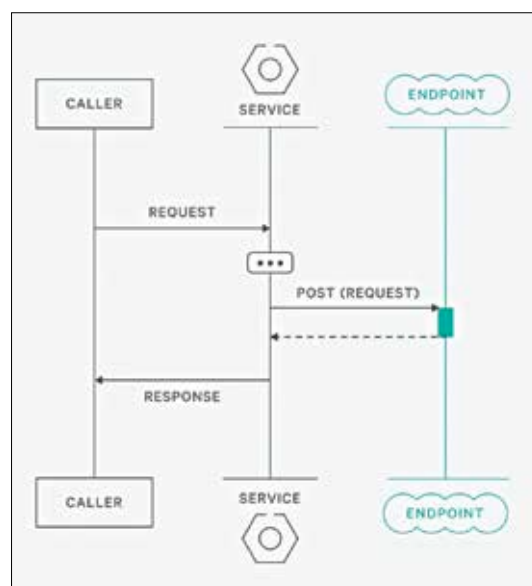
One of the core principles of Ballerina is to make use of sequence diagrams to write code. When dealing with distributed computing, sequence diagrams are the norm to figuring out when, how and who will be speaking to which module.

It's when you get around to translating your sequence diagrams into code that all hell breaks loose. With Ballerina, the sequence diagrams are the code. And it works vice versa as well. This approach to writing code requires that textual and graphical syntaxes have a 1:1 parity so that your code does exactly what it's designed to do and there aren't multiple outcomes to a particular sequence diagram or vice versa.

The other key facet about Ballerina is that all the networking data types are clearly visible to the code at all times. The usual approach involves utilising multiple modules to ensure

communication within your serverless application and the network information is hidden within these modules. With Ballerina, the network is visible from the get go, so that your code can actually make use of the various network data types.

When there are errors found in the code, Ballerina will not allow those errors to be executed. This is why Ballerina is termed as being opinionated. While this approach ensures fewer vulnerabilities to exist, it also requires that you, firstly, write and implement good code.



Your diagram is the code

```
if (result is error) {
    io:println("Error in responding: ", result);
}
}
```

Save the file as hello_world.bal and run it from the command line using:

```
$ ballerina run hello_world.bal
```

And your output should be:

```
[ballerina/http] started HTTP/WS listener 0.0.0.0:9090
```

Now that the service is up and running, you can call it using any HTTP

*CODE WATCH



*Why Ballerina?

Paul Fremantle, CTO WSQ2 speaks about the motivation for creating a new language.

<https://dgit.in/devnov19-1>



*Creating a WebSocket

Creating a basic WebSocket in Ballerina couldn't have been simpler than this.

<https://dgit.in/devnov19-2>



*Dockerina

Ballerina can understand and help orchestrate microservices into Kubernetes and Docker.

<https://dgit.in/devnov19-3>